Week 4 Python Learning Guide

Python foundations for Week 4. The examples teach the skills without completing the assignment functions.

1. Variables and basic Python values

A variable stores information. You can store text, whole numbers, decimals, True or False, and a missing value called None.

```
item = "notebook"       # text (str)
price = 4               # whole number (int)
tax = 0.08              # decimal (float)
available = True        # Boolean (bool)
delivery_date = None    # no value yet

print(item)
print(price * (1 + tax))
```

What the code means: The = sign saves a value. print() displays it. None is not the same as 0 or an empty string.

2. Lists, dictionaries, tuples and sets

Python uses different containers to organise information. A list is ordered, a dictionary stores key-value pairs, a tuple groups fixed values, and a set removes duplicates.

```
prices = [2, 4, 6]
print(prices[0])             # 2
print(len(prices))           # 3

product = {"code": 101, "cost": 4}
print(product["cost"])      # 4
print(product.get("stock")) # None if key is absent

fields = ("code", "cost")  # tuple
unique = set([1, 1, 3])    # {1, 3}
```

What the code means: Use square brackets to read a list position or dictionary key. The .get() method can read a key without raising an error when it is missing. Sets contain each value once.

3. Conditions, loops and missing values

An if statement checks a condition. A for loop repeats code. These let a program inspect many items and make a decision for each one.

```
measurements = [12, None, 15, 9]

for value in measurements:
    if value is None:
        print("Missing measurement")
    elif value >= 12:
        print("High")
    else:
        print("Low")
```

What the code means: Use is None to check for a missing value. The == operator compares values; >= means greater than or equal to. Indentation shows which lines belong to each condition.

4. Functions, inputs and return values

A function is reusable code. Parameters are the inputs it accepts. return sends the result back to the rest of the program.

```
def calculate_discount(cost, percent):
    saved = cost * percent / 100
    return cost - saved

new_cost = calculate_discount(40, 25)
print(new_cost)  # 30.0

def split_total(total):
    return total // 2, total % 2

whole, left_over = split_total(9)
print(whole, left_over)
```

What the code means: A function does not run just because it is defined. You must call it. Python can return more than one value, which you can store in separate variables. pass is only a placeholder and does not perform a task.

5. Copies, percentages and random selection

A copy lets you change a dictionary without changing the original one. A percentage gives part of a total. Random sampling chooses items without repeating them.

```
settings = {"volume": 5}
new_settings = settings.copy()
new_settings["volume"] = 8
print(settings["volume"])  # still 5

items = [101, 102, 103, 104, 105]
import random
rng = random.Random(42)
picked = rng.sample(items, 2)
print(picked)

print(round(20 * 30 / 100))  # 6
```

What the code means: The .copy() method makes a separate dictionary. random.Random(42) creates a reproducible random generator. sample(list, count) selects distinct entries. round() rounds a number; check how it behaves at .5 values.

6. Sets, membership and continue

A set is useful when you only care whether an ID belongs to a group. continue skips the rest of the current loop turn but does not stop the entire loop.

```
closed_locations = {"room_b", "room_d"}
locations = ["room_a", "room_b", "room_c", "room_d"]

for location in locations:
    if location in closed_locations:
        continue
    print("Open:", location)
```

What the code means: The in operator checks membership. Here, the program skips printing closed locations and keeps processing the rest. Do not put unrelated work inside a skipped block if it must still happen.

7. Dropdowns, sliders and changing output

ipywidgets creates controls in Jupyter notebooks. A dropdown lets the user choose an option. A slider stores a number. A callback function runs when a control changes.

```
import ipywidgets as widgets
from IPython.display import display

quantity = widgets.IntSlider(
    value=1, min=1, max=10, description="Quantity"
```

```
)
output = widgets.Output()

def refresh(change=None):
    with output:
        output.clear_output(wait=True)
        print("Quantity:", quantity.value)

quantity.observe(refresh, names="value")
display(quantity, output)
refresh()
```

What the code means: The .value property holds the current control setting. observe() connects a change to a function. Output() provides a place to display results. clear_output() removes the old display before showing the new one.

8. Graphs, checks and CSV files

Matplotlib draws graphs. Tests help catch mistakes. Pandas stores data in tables and can save a CSV file.

```
import matplotlib.pyplot as plt
import pandas as pd

days = [0, 1, 2, 3]
sales = [2, 4, 3, 6]

assert len(days) == len(sales)
assert all(value >= 0 for value in sales)

plt.plot(days, sales, label="Sales")
plt.xlabel("Day")
plt.ylabel("Items sold")
plt.legend()
plt.show()

record = {"category": "practice", "peak": max(sales)}
pd.DataFrame([record]).to_csv("practice.csv", index=False)
```

What the code means: len() checks the number of values, all() checks that every condition is true, max() finds the largest value, and assert reports a failed check. A CSV is a table saved as plain text.

Reference links

Python tutorial: https://docs.python.org/3/tutorial/

Python random module: https://docs.python.org/3/library/random.html

ipywidgets documentation: https://ipywidgets.readthedocs.io/en/stable/

Matplotlib pyplot: https://matplotlib.org/stable/tutorials/pyplot.html

Pandas DataFrame: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html